Ejercicios modelo de Bases de Datos (parcial 2)

Selección de los ejercicios más representativos de las prácticas que pasaste, resueltos paso a paso. Cada uno indica a qué tema y a qué tipo de pregunta de los parciales viejos se parece.

#	Ejercicio	Tema	Se parece a
1	Almacenamiento 1, 2, 7 y 8	Tamaños, bloques, claves por nodo, altura B+	Recuperatorio 18/11/2025 (altura y bloques)
2	Evaluación 1 (pinturas)	Materialización con y sin índice	Recuperatorio 21/11/2023 (ventas)
3	Evaluación 3 (b y c)	Costos con variables, concatenación y agregación	Recuperatorio 21/11/2023 (agrupación)
4	Álgebra 6	Costo de reuniones	Parciales con reunión y índices
5	Álgebra 2	Selecciones con negación y índice B+	Pregunta conceptual de selección
6	Optimización 4	Programación dinámica con 4 tablas	Parcial 06/11/2025, ejercicio 2
7	Optimización 5	Programación dinámica y heurísticas juntas	Parcial 06/11/2025, ejercicio 2
8	Optimización 3.3	Heurísticas paso a paso	Preguntas de optimización heurística
9	Recuperación de información 3, 5 y 6	TF-IDF, DIFF y NOT, al menos k de n palabras	Parcial 06/11/2025, ejercicio 3

---

1. Tamaños, bloques y árbol B+ (Almacenamiento 1, 2, 7 y 8)

Enunciado. curso(nombre-curso, aula, nombre-profesor) tiene 300 registros. matrícula(nombre-curso, nombre-estudiante, nivel, número-matrícula) tiene 30.000. Archivos secuenciales, bloques de 4096 bytes. Hay un índice primario B+ en matrícula.

El enunciado no da el tamaño de los atributos, así que hay que **suponerlo y decirlo**. Supongo: nombres de curso, profesor y estudiante 30 bytes; aula 10 bytes; nivel y número-matrícula 4 bytes; puntero 8 bytes.

Ej 1: tamaño de un registro (suma de atributos más el puntero al siguiente).

- curso: 30 + 10 + 30 + 8 = **78 bytes**.
- matrícula: 30 + 30 + 4 + 4 + 8 = **76 bytes**.

Ej 2: bloques. Con la fórmula del material, bloques = ⌈N · R / B⌉.

- curso: ⌈300 · 78 / 4096⌉ = ⌈23.400 / 4096⌉ = ⌈5,71⌉ = **6 bloques**.
- matrícula: ⌈30.000 · 76 / 4096⌉ = ⌈2.280.000 / 4096⌉ = ⌈556,6⌉ = **557 bloques**.

Ej 7: claves de búsqueda por nodo. La clave es el número de matrícula (el enunciado dice nombre de matrícula, que interpreto como ese campo): TCB = 4 bytes. De C = n · TCB + (n + 1) · 8 sale n = (C − 8) / (TCB + 8).

n = (4096 − 8) / (4 + 8) = 4088 / 12 = 340,67, así que entran **340 claves** por nodo (341 punteros).

Ej 8: altura. K = 30.000 valores distintos de clave (el número de matrícula es único). Con n = 341 punteros, ⌈n/2⌉ = 171.

$h \leqslant \lceil \log_{171}(30.000) \rceil = \lceil \ln 30.000 / \ln 171 \rceil = \lceil 10,31 / 5,14 \rceil = \lceil 2,005 \rceil =$ **3**.

Comprobación: $171^2 = 29.241$, que está apenas por debajo de 30.000, así que dos niveles no alcanzan y hacen falta tres. Con n = 340 también da 3.

---

2. Materialización con y sin índice (Evaluación 1, pinturas)

Enunciado. pinturas tiene 10.000 registros, 10 por bloque. Entran 100 registros con solo color y precio. 3/4 de las pinturas tienen precio ⩾ 1000. Se comparan dos árboles y tres escenarios de índice en precio (primario, secundario, ninguno). Una tabla temporal **no tiene índices**.

Datos previos.

- Bloques de pinturas: 10.000 / 10 = **1000**.
- fs(precio < 1000) = 1 − 3/4 = **1/4**, así que pasan 2500 tuplas.
- Tuplas completas: 10 por bloque. Tuplas reducidas (color, precio): 100 por bloque.

Árbol a: σ precio < 1000 (π color, precio (pinturas))

Nodo	Costo del operador	Salida	Se escribe
π color, precio	1000 (recorre la tabla)	10.000 tuplas reducidas = 100 bloques	100
σ precio < 1000	100 (lee la temporal, sin índice)	2500 tuplas reducidas = 25 bloques	0 (resultado final)

Total = 1000 + 100 + 100 = 1200, en los tres escenarios. El índice en precio no ayuda porque la selección trabaja sobre la tabla temporal, que no tiene índices.

Árbol b: π color, precio (σ precio < 1000 (pinturas))

La selección deja 2500 tuplas **completas**, que ocupan 2500 / 10 = **250 bloques**. Se escriben 250. La proyección las lee (250) y deja 2500 tuplas reducidas = 25 bloques (final, no se escribe). Lo que cambia es el costo de la selección:

- **Índice primario en precio.** La tabla está ordenada por precio, así que las tuplas con precio < 1000 son las primeras. No se usa el árbol: se escanea desde el principio hasta la primera tupla con precio ⩾ 1000. Costo **250**.
- **Índice secundario en precio.** Se podrían recorrer las hojas desde el principio y buscar cada tupla, pero en el peor caso son unos 2500 accesos a bloque (uno por registro), más que los 1000 del recorrido secuencial. El optimizador elige el **recorrido secuencial: 1000**.
- **Sin índice.** Recorrido secuencial: **1000**.

Escenario	Costo de σ	Total (σ + escribir 250 + π lee 250)
Primario	250	750
Secundario	1000	1500
Ninguno	1000	1500

Conclusión

	Árbol a (π primero)	Árbol b (σ primero)
Primario	1200	**750**
Secundario	**1200**	1500
Ninguno	**1200**	1500

La regla de empujar la selección hacia abajo **no siempre gana**. Si no hay un índice útil, proyectar primero achica la tabla 10 veces (100 bloques) y la temporal sale mucho más barata. Por eso el optimizador compara costos en lugar de aplicar la heurística a ciegas.

---

3. Costos con variables (Evaluación 3, partes b y c)

La tabla es acceso_dispositivo(usuario, fecha, dispositivo) con **índice primario en usuario** (está ordenada por usuario y un usuario aparece en muchas filas). Sea hi la altura del índice y br los bloques de la tabla.

b) σ usuario = Raúl (acceso) ++ σ usuario = Luisa (acceso)

Sean bR y bL los bloques que ocupan las tuplas de Raúl y de Luisa.

- Primer σ: busca por el índice primario (igualdad, no clave): hi + bR. Escribe bR bloques.
- Segundo σ: hi + bL. Escribe bL bloques.
- La concatenación lee ambas tablas temporales completas: bR + bL. Su resultado es el final y no se escribe.

Total = (hi + bR) + (hi + bL) + bR + bL + (bR + bL) = 2·hi + 3·(bR + bL).

c) usuario γ count(dispositivo) (acceso)

Como hay índice primario en usuario, la tabla **ya está ordenada por usuario** y los grupos son contiguos. No hace falta ordenar: se recorre la tabla una sola vez y se cuenta al vuelo. Costo **br** transferencias. Si la tabla no estuviera ordenada habría que sumar un ordenamiento externo, br · (2k + 1).

---

4. Costo de reuniones (Álgebra 6)

Enunciado. r1(A,B,C) tiene 20.000 tuplas, 25 por bloque. r2(C,D,E) tiene 45.000 tuplas, 30 por bloque. Estimar transferencias y accesos para r1 ⋈ r2 con bucle anidado por bloques y con mezcla, y cuántos bloques se escriben.

Bloques. br1 = 20.000 / 25 = **800**. br2 = 45.000 / 30 = **1500**. Sea bf los bloques del resultado.

a) Bucle anidado por bloques. Conviene la tabla chica como externa (r1):

- Transferencias = br · bs + br + bf = 800 · 1500 + 800 + bf = **1.200.800 + bf**.
- Accesos (con bloques contiguos) = 2 · br + 2 · bf = **1600 + 2·bf**.
- Si se usara r2 como externa saldría 1500 · 800 + 1500 = 1.201.500 + bf, un poco peor.

b) Reunión por mezcla (suponiendo ambas ordenadas por C):

- Transferencias = br + bs + bf = 800 + 1500 + bf = **2300 + bf**.
- Accesos = br + bs + bf = **2300 + bf**.
- Si no estuvieran ordenadas, se suma el costo de ordenar cada una: br · (2k + 1) por tabla, que depende de M.

La diferencia es enorme: unos 2300 contra más de 1.200.000.

¿Cuántos bloques se escriben? ¿Cambia con la estrategia? Se escriben **bf bloques**, y **no cambia con la estrategia**: los dos algoritmos producen el mismo resultado, así que ocupa lo mismo. Lo que cambia es el costo de leer. bf se estima con fs de reunión, 1 / max(V(C, r1), V(C, r2)), que el enunciado no da, así que queda simbólico.

---

5. Selecciones con negación (Álgebra 2)

Tabla sucursal(nombre-sucursal, ciudad-sucursal, activos) con un índice B+ solo en ciudad-sucursal. La idea es **reescribir la negación** para ver qué tipo de condición queda.

a) σ not(ciudad < Arganzuela) equivale a ciudad ≥ Arganzuela. Es una condición de rango, así que **se usa el índice**: se busca el primer valor ≥ Arganzuela en el árbol y se recorren las hojas hacia adelante (si el índice es primario, las tuplas están contiguas y se lee un tramo del archivo; si es secundario, cada tupla puede costar un bloque).

b) σ not(ciudad = Arganzuela) equivale a ciudad ≠ Arganzuela. Casi todas las tuplas cumplen la condición, así que el índice no sirve (habría que visitar casi toda la tabla). Lo mejor es el **recorrido secuencial**.

c) σ not(ciudad < Arganzuela o activos < 5000). Por De Morgan: ciudad ⩾ Arganzuela **y** activos ⩾ 5000. Se usa el índice para ciudad ⩾ Arganzuela (la única condición con índice) y sobre las tuplas recuperadas se **filtra en memoria** activos ⩾ 5000. Si ciudad ⩾ Arganzuela no es selectiva (pasa casi toda la tabla), conviene el recorrido secuencial.

---

6. Programación dinámica con 4 tablas (Optimización 4)

W(a,b) 400 tuplas, X(b,c) 300, Y(c,d) 200, Z(d,e) 100. V(W,a) = 50, V(W,b) = 40, V(X,b) = 60, V(X,c) = 100, V(Y,c) = 50, V(Y,d) = 20, V(Z,d) = 10, V(Z,e) = 50. Regla: el costo de una partición suma los mejores costos de los dos grupos y los tamaños de los que son resultados intermedios (una tabla base no suma).

Pares (costo 0): {W,X} por b = 2000. {X,Y} por c = 600. {Y,Z} por d = 1000. {W,Y} = 80.000, {W,Z} = 40.000 y {X,Z} = 30.000 son producto cartesiano.

Tríos:

Conjunto	Ganadora	Costo	Tamaño
{W,X,Y}	(X ⋈ Y) ⋈ W	600	4000
{W,X,Z}	(W ⋈ X) ⋈ Z	2000	200.000
{W,Y,Z}	(Y ⋈ Z) ⋈ W	1000	400.000
{X,Y,Z}	(X ⋈ Y) ⋈ Z	600	3000

Las cuatro tablas (tamaño 20.000). Las particiones 2+2 suman los dos tamaños de los pares. Las 3+1 suman el costo del trío más su tamaño:

- {{W,X},{Y,Z}}: 2000 + 1000 = **3000**
- {{W,Z},{X,Y}}: 40.000 + 600 = 40.600
- {{W,Y},{X,Z}}: 80.000 + 30.000 = 110.000
- {{X,Y,Z},{W}}: 600 + 3000 = **3600**
- {{W,X,Y},{Z}}: 600 + 4000 = 4600
- {{W,X,Z},{Y}}: 2000 + 200.000 = 202.000
- {{W,Y,Z},{X}}: 1000 + 400.000 = 401.000

Respuestas.

1. **Solo árboles profundos a la izquierda** (solo particiones 3+1): el mejor plan es **((X ⋈ Y) ⋈ Z) ⋈ W**, con costo **3600**.
2. **Todos los árboles** (también 2+2): el mejor plan es **(W ⋈ X) ⋈ (Y ⋈ Z)**, con costo **3000**.

Un árbol profundo a la izquierda tiene siempre una tabla base como lado derecho de cada reunión. Buscar solo ahí ahorra trabajo al optimizador, pero puede perder el plan óptimo, como pasa acá.

---

7. Programación dinámica y heurísticas (Optimización 5)

Consulta. Π nombre, DNI, nombreBib (σ antigüedad > 5 y rol = bibliotecario (persona ⋈ bibliotecario ⋈ trabajaEn)).

Datos. persona 1000 tuplas, bibliotecario 400, trabajaEn 1500. V(trabajaEn, DNI) = 750. El DNI es clave en persona y en bibliotecario, así que V(persona, DNI) = 1000 y V(bibliotecario, DNI) = 400 (supuesto razonable, el enunciado no lo da).

Paso 1: orden de las reuniones (programación dinámica)

Las tres tablas comparten DNI. Pares (costo 0):

- {persona, bibliotecario}: 1000 · 400 / max(1000, 400) = **400**.
- {persona, trabajaEn}: 1000 · 1500 / max(1000, 750) = **1500**.
- {bibliotecario, trabajaEn}: 400 · 1500 / max(400, 750) = **800**.

Trío (el costo es el tamaño del par):

- {persona, bibliotecario} + trabajaEn: **400**
- {bibliotecario, trabajaEn} + persona: 800
- {persona, trabajaEn} + bibliotecario: 1500

La ganadora es la primera: **(persona ⋈ bibliotecario) ⋈ trabajaEn**, con costo 400.

Paso 2: heurísticas sobre esa consulta

Partiendo de Π (σ antigüedad > 5 y rol = bibliotecario ((persona ⋈ bibliotecario) ⋈ trabajaEn)):

1. **Partir la selección con AND** en dos selecciones sucesivas.
2. **Empujar cada selección** hacia la tabla que tiene su atributo: antigüedad es solo de bibliotecario y rol es solo de trabajaEn.

3. **Empujar las proyecciones**, dejando en cada tabla solo lo que se necesita más arriba (los atributos de salida y los de reunión): persona necesita DNI y nombre, bibliotecario solo DNI, trabajaEn DNI y nombreBib.

Resultado:

Π nombre, DNI, nombreBib ((Π DNI, nombre (persona) ⋈ Π DNI (σ antigüedad > 5 (bibliotecario))) ⋈ Π DNI, nombreBib (σ rol = bibliotecario (trabajaEn)))

Se mantiene el orden de reuniones del paso 1, y como piden el orden por costo antes que la heurística, no se vuelve a calcular con los tamaños achicados.

---

8. Heurísticas paso a paso (Optimización 3.3)

Consulta. Π name, title (σ dept_name = Comp. Sci. (instructor ⋈ teaches ⋈ course)).

Paso 1: selección temprana. dept_name es un atributo solo de instructor, así que la selección se empuja adentro de la reunión (la selección conmuta con la reunión natural cuando solo usa atributos de un lado):

Π name, title ((σ dept_name = Comp. Sci. (instructor) ⋈ teaches) ⋈ course)

Paso 2: proyección temprana. Cada tabla conserva solo los atributos que se usan arriba: instructor necesita ID y name, teaches necesita ID y course_id, course necesita course_id y title. Los atributos de reunión (ID y course_id) se mantienen:

Π name, title ((Π ID, name (σ dept_name = Comp. Sci. (instructor)) ⋈ Π ID, course_id (teaches)) ⋈ Π course_id, title (course))

Reglas de equivalencia usadas, en orden:

1. Conmutar selección con reunión natural (atributos de un solo lado).
2. Empujar proyección sobre reunión natural (conservando los atributos de reunión).
3. Asociatividad de la reunión para fijar el orden de agrupación.

---

9. Recuperación de información (práctica 3, 5 y 6)

Ej 3: TF-IDF si un término está en todos los documentos, o en uno solo

IDF = log(N / n_i), con N documentos en total y n_i los documentos donde aparece el término.

- **En todos los documentos:** n_i = N, así que IDF = log(1) = **0** y TF-IDF = 0 en todos. El término no distingue un documento de otro (se comporta como una stop word).
- **En un solo documento:** n_i = 1, así que IDF = log(N), el **valor máximo**. Ese término discrimina fuertemente y el documento que lo contiene sube en el ranking (más cuanto mayor sea su TF).

Ej 5: operador DIFF

Definición lógica. A DIFF B = A y no B: los documentos de A que no están en B. Con S_A y S_B los conjuntos de documentos: **S_A − S_B**.

Implementación con índice invertido. Las listas invertidas vienen ordenadas por id de documento. Se recorren las dos en paralelo con un puntero cada una: si el id actual de A es menor que el de B, va al resultado y avanza A. Si es igual, se descarta y avanzan ambas. Si es mayor, avanza B. Al terminar B se copia el resto de A. Cuesta O(|A| + |B|), una pasada por cada lista.

NOT como DIFF. NOT t = TODOS DIFF t, donde TODOS es la lista de todos los documentos de la colección: **NOT t = S − S_t**.

De regalo, XOR con DIFF: A XOR B = (A DIFF B) ∪ (B DIFF A).

Ej 6: documentos con al menos k de n palabras clave

Dato: cada palabra tiene su lista ordenada de ids. La idea es una **mezcla de las n listas** contando cuántas listas contienen cada id:

```
montar un heap con el primer id de cada una de las n listas
mientras el heap no esté vacío:
    id = menor id del heap; cuenta = 0
    mientras el menor del heap sea id:
        sacarlo; cuenta = cuenta + 1
        insertar el siguiente id de la misma lista
    si cuenta >= k: agregar id al resultado
```

Cada lista se recorre una sola vez y el costo es **O(L log n)**, con L la suma de los largos de las listas. Casos límite: k = 1 es un OR (unión) y k = n es un AND (intersección).

Lo que dejé afuera, y por qué

- **Recuperación 12 (shards y coordinador)** es un ejercicio largo de integración que no aparece en los parciales viejos que pasaste.
- **Recuperación 10 y 11 (PageRank y rastreador web)** y **9 (PC contra web)**: dependen de material que no está en las filminas que leí, así que no quise inventar la respuesta.
- **Recuperación 7 (proximidad ADJ, PRE y NEAR)**, **Álgebra 3, 4, 5, 7 y 8**, y **Optimización 1 y 2** (demostraciones con reglas de equivalencia): son implementaciones o demostraciones. Si querés alguno, decime cuál.